Simulación computacional de un proceso de guerra de guerrillas

Juan Esteban Gómez Gómez, Jose Luis Martinez Diaz, William David Rincón Verano, Serena Triviño Sáenz
Universidad Nacional de Colombia
(30 de octubre de 2026)

En este trabajo se estudia mediante simulación computacional un modelo estocástico de fragmentación y fusión aplicado a un proceso de guerra de guerrillas. El sistema se representa mediante unidades de ataque que interactúan de acuerdo con reglas probabilísticas, conservando una fuerza total de $N = 30\,000$. Se desarrollaron dos implementaciones independientes, en C++ y Python, para obtener las distribuciones de tamaños y analizar su comportamiento frente a una ley de potencia de referencia, caracterizada por el exponente teórico $\alpha = 2{,}5$. Asimismo, se examinó la influencia del parámetro de fragmentación ν para cuatro valores entre 0,005 y 0,1. Los resultados muestran distribuciones decrecientes compatibles con un comportamiento de ley de potencia y exponentes estimados entre 2,514 y 2,634 en Python, y entre 2,5686 y 2,5953 en C++. La comparación entre ambas implementaciones presentó diferencias relativas inferiores al 2,2 %. Finalmente, se verificó la conservación de la fuerza total en Python y se compararon los tiempos de ejecución, observándose un menor tiempo registrado para C++ bajo los procedimientos computacionales utilizados.

Palabras clave: fragmentación, fusión, simulación estocástica, guerra de guerrillas, ley de potencia.

I. INTRODUCCIÓN

Los sistemas constituidos por múltiples unidades que interactúan mediante procesos de fragmentación y fusión pueden presentar distribuciones estadísticas de tamaño que siguen leyes de potencia. El estudio de estos sistemas permite analizar cómo reglas locales y probabilísticas generan comportamientos colectivos a escala global. Johnson et al. [1] estudiaron regularidades estadísticas en conflictos armados y propusieron un marco para analizar la distribución de la intensidad de los eventos violentos. Posteriormente, Johnson et al. [2] examinaron patrones estadísticos comunes en guerras y terrorismo, proporcionando un contexto para el estudio de modelos de fragmentación y fusión.

En este trabajo se estudia un modelo estocástico en el que la fuerza total del sistema se distribuye entre diferentes unidades de ataque. Estas unidades pueden fragmentarse en componentes de tamaño unitario o fusionarse con otras unidades, de acuerdo con reglas probabilísticas. La evolución del sistema se simula computacionalmente siguiendo el modelo descrito en los artículos de referencia.

El objetivo es obtener y analizar la distribución de tamaños de las unidades, contrastarla con la ley de potencia teórica $n_s \propto s^{-\alpha}$, cuyo exponente de referencia es $\alpha = 2{,}5$, y estudiar cómo varía la distribución al modificar el parámetro de fragmentación ν. Para ello, se comparan dos implementaciones independientes, desarrolladas en C++ y Python, considerando tanto los resultados estadísticos como sus tiempos de ejecución.

II. OBJETIVOS

A. Objetivo general

Implementar y analizar computacionalmente un modelo estocástico de fragmentación y fusión aplicado a un proceso de guerra de guerrillas, con el propósito de estudiar la distribución de tamaños de las unidades, contrastarla con la ley de potencia teórica de exponente $\alpha = 2{,}5$ y comparar los resultados y el desempeño computacional de dos implementaciones desarrolladas en C++ y Python.

B. Objetivos específicos

- Implementar las reglas probabilísticas de selección, fragmentación y fusión mediante algoritmos desarrollados en C++ y Python.
- Verificar la conservación de la fuerza total del sistema durante la evolución de la simulación.
- Obtener y analizar la distribución estadística del número de unidades en función de su tamaño, y contrastarla con el comportamiento teórico esperado.
- Estimar el exponente α mediante un ajuste lineal en escala logarítmica y compararlo con el valor teórico de referencia.
- Examinar la variación de las distribuciones de tamaños y del exponente estimado al modificar el parámetro de fragmentación ν.
- Comparar los resultados estadísticos y los tiempos de ejecución de las implementaciones en C++ y Python, considerando las condiciones de simulación utilizadas.

III. MODELO MATEMÁTICO

A. Representación del sistema

El sistema está constituido por un conjunto de unidades, cada una caracterizada por un tamaño o fuerza de ataque s_i. La fuerza total del sistema se conserva durante la evolución y se expresa como

$$\sum_{i=1}^{M} s_i = N, \tag{1}$$

donde N representa la fuerza total del sistema y M es el número de unidades presentes en un instante determinado.

Inicialmente, el sistema se compone de N unidades de tamaño unitario, de modo que $s_i = 1$ para todas ellas. Las interacciones modifican el número de unidades y sus tamaños, sin alterar la suma total de sus tamaños.

B. Selección de unidades

La selección de una unidad depende de su tamaño. La probabilidad de seleccionar la unidad i se define como

$$P_i = \frac{s_i}{N}. \tag{2}$$

Por tanto, las unidades de mayor tamaño tienen una probabilidad proporcionalmente mayor de ser seleccionadas. Esta regla introduce una dependencia entre el tamaño de cada unidad y su participación en las interacciones del sistema.

C. Fragmentación

Con probabilidad ν, la unidad seleccionada se fragmenta en s_i unidades de tamaño unitario. Si la unidad original tiene tamaño s_i, la fragmentación conserva la fuerza total, puesto que

$$s_i = \underbrace{1 + 1 + \cdots + 1}_{s_i \text{ unidades}}. \tag{3}$$

Este proceso incrementa el número de unidades y distribuye la fuerza entre componentes de menor tamaño.

D. Fusión

Con probabilidad $1-\nu$, se selecciona otra unidad de tamaño s_j y ambas se fusionan en una unidad cuyo tamaño es

$$s_{\text{nueva}} = s_i + s_j. \tag{4}$$

La fusión reduce el número de unidades y concentra la fuerza en componentes de mayor tamaño. La selección de la segunda unidad se realiza de acuerdo con la regla probabilística implementada en el algoritmo.

E. Distribución de tamaños y exponente

Sea n_s el número de unidades de tamaño s. La distribución de tamaños permite caracterizar el estado estadístico del sistema. Como referencia teórica, se considera una distribución de ley de potencia de la forma

$$n_s \propto s^{-\alpha}, \tag{5}$$

donde α es el exponente que caracteriza el decrecimiento de la distribución.

Al tomar logaritmos en ambos lados de la ecuación (5), se obtiene una relación lineal:

$$\log(n_s) = C - \alpha \log(s), \tag{6}$$

donde C es una constante. Por tanto, el exponente se estima como el negativo de la pendiente del ajuste lineal entre $\log(s)$ y $\log(n_s)$.

Para el análisis se utiliza $\alpha = 2{,}5$ como valor teórico de referencia. La comparación con este valor se realiza mediante las distribuciones obtenidas y los exponentes estimados numéricamente.

IV. ALGORITMO E IMPLEMENTACIÓN COMPUTACIONAL

A. Procedimiento de simulación

El modelo se implementó de manera independiente en C++ y Python. Ambas implementaciones representan los procesos de selección, fragmentación y fusión de las unidades, con el propósito de estudiar la evolución del sistema y obtener la distribución de tamaños de las unidades de ataque.

El procedimiento general consiste en inicializar el sistema con N unidades de tamaño uno, seleccionar unidades de acuerdo con la regla probabilística del modelo y decidir si ocurre fragmentación o fusión mediante el parámetro ν. Después de cada interacción se actualiza el estado del sistema y se repite el proceso durante el número de pasos establecido. Finalmente, se obtiene la distribución de tamaños y se estima el exponente de escala.

Las implementaciones difieren en la forma de procesar los datos: Python analiza la distribución del estado final de la simulación, mientras que C++ acumula muestras durante la ejecución. Esta diferencia debe tenerse en cuenta al comparar las distribuciones y los exponentes estimados.

B. Organización y gestión del trabajo colaborativo

Para coordinar el desarrollo del proyecto se utilizó la plataforma *Jira*, con el propósito de organizar las actividades, asignar responsabilidades y realizar seguimiento al avance del equipo de trabajo. Mediante un tablero colaborativo se distribuyeron las tareas relacionadas con la revisión del modelo matemático, la implementación computacional, el análisis de resultados y la elaboración del informe.

Esta herramienta permitió visualizar las actividades pendientes, en ejecución y completadas, facilitando la coordinación entre los integrantes y el seguimiento de los compromisos establecidos. Como evidencia del proceso de organización se presenta el tablero utilizado durante el desarrollo del proyecto.

El espacio de trabajo puede consultarse en el siguiente enlace: `https://unal-team-bomb75.atlassian.net/jira/software/projects/KAN/list?jql=project%20%3D%20KAN%20ORDER%20BY%20cf%5B10019%5D%20ASC`.

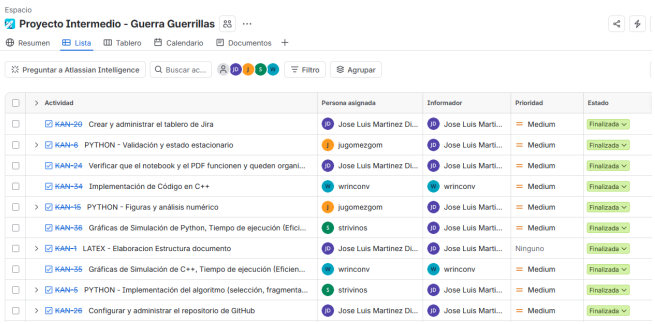


Figura 1. Tablero de Jira utilizado para la asignación, organización y seguimiento de las actividades del proyecto.

C. Parámetros de simulación

En las simulaciones finales se utilizó una fuerza total $N = 30\,000$ y un parámetro de fragmentación $\nu = 0{,}01$ para el análisis principal. También se estudiaron cuatro valores del parámetro de fragmentación: $\nu \in \{0{,}005,\ 0{,}01,\ 0{,}05,\ 0{,}1\}$.

La Tabla I resume los parámetros principales de las dos implementaciones.

Cuadro I. Parámetros de las simulaciones principales.

Parámetro	Símbolo	Valor
Fuerza total	N	30 000
Fragmentación	ν	0,01
Pasos de Python	T_{Py}	9 000 000
Pasos de C++	$T_{\mathrm{C++}}$	900 000 000
Semilla inicial de C++	–	6767
Valores de ν	–	0,005, 0,01, 0,05, 0,1

En Python, la simulación principal se ejecutó durante 300 barridos, equivalentes a 9 000 000 eventos para $N = 30\,000$. En C++, la ejecución principal utilizó 900 000 000 pasos. Para el barrido de ν, se consideraron los cuatro valores indicados. El número de pasos y el procedimiento de acumulación de muestras no son iguales en ambas implementaciones, por lo que estas diferencias se consideran al interpretar los resultados.

D. Estimación del exponente

El exponente α se estimó mediante un ajuste lineal de $\log_{10}(n_s)$ frente a $\log_{10}(s)$, considerando los valores positivos de n_s en el intervalo

$$2 \le s \le 30. \tag{7}$$

Si m representa la pendiente obtenida mediante regresión lineal, el exponente estimado se calcula como

$$\widehat{\alpha} = -m. \tag{8}$$

Este procedimiento proporciona una estimación exploratoria del exponente. Su valor depende de la distribución obtenida, del intervalo de ajuste y del procedimiento utilizado para acumular los datos. Por ello, los resultados se comparan con el valor teórico de referencia $\alpha = 2{,}5$ sin asumir una coincidencia exacta.

V. RESULTADOS

A. Distribución de tamaños

Las simulaciones permiten obtener la distribución del número de unidades n_s en función de su tamaño s. Para el caso principal, se utilizó una fuerza total $N = 30\,000$ y un parámetro de fragmentación $\nu = 0{,}01$. Las distribuciones obtenidas mediante las implementaciones en C++ y Python se presentan en las Figuras 2 y 3, respectivamente.

En ambas implementaciones se observa que el número de unidades disminuye al aumentar su tamaño, comportamiento compatible con una distribución de tipo ley de potencia. Sin embargo, las distribuciones presentan diferencias, por lo que la comparación cuantitativa se realiza mediante los exponentes estimados.

B. Efecto del parámetro de fragmentación

Para examinar la influencia de la fragmentación sobre la distribución de tamaños, se consideraron cuatro valores del parámetro: $\nu = 0{,}005$, 0,01, 0,05 y 0,1. Los resultados obtenidos mediante las implementaciones en C++ y Python se presentan en las Figuras 4 y 5, respectivamente.

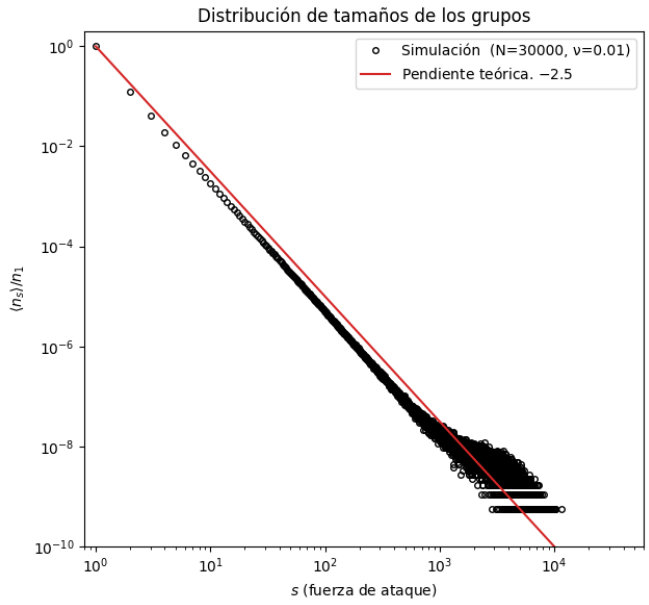


Figura 2. Distribución del número de unidades en función de su tamaño, obtenida mediante la implementación en C++ para $N = 30\,000$ y $\nu = 0{,}01$.

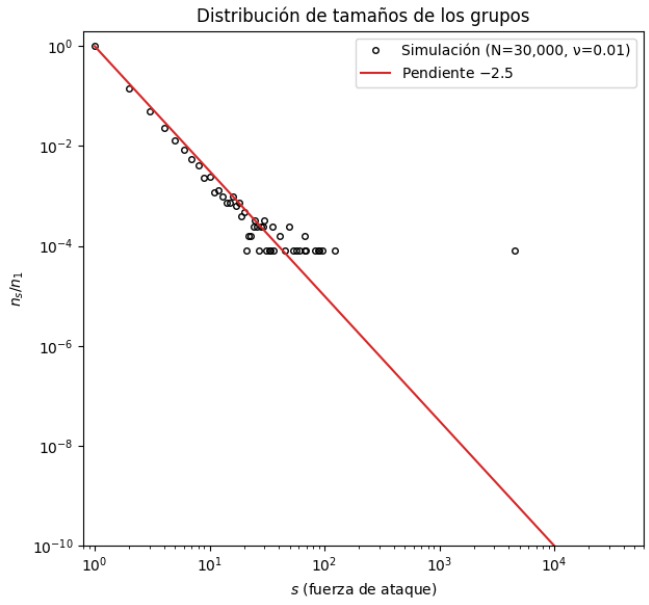


Figura 3. Distribución del número de unidades en función de su tamaño, obtenida mediante la implementación en Python para $N = 30\,000$ y $\nu = 0{,}01$.

Los resultados permiten comparar cómo cambia la distribución al modificar la probabilidad de fragmentación. La magnitud de estos cambios se evalúa también mediante el exponente α estimado para cada valor de ν.

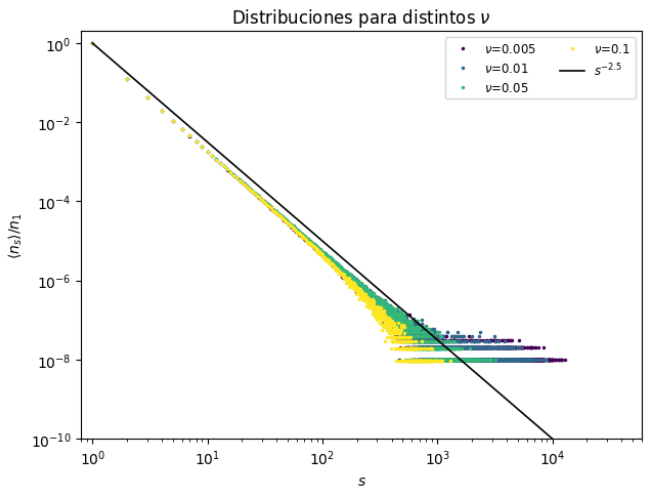


Figura 4. Distribuciones del número de unidades en función de su tamaño, obtenidas mediante C++ para los valores del parámetro de fragmentación $\nu \in \{0{,}005,\ 0{,}01,\ 0{,}05,\ 0{,}1\}$.

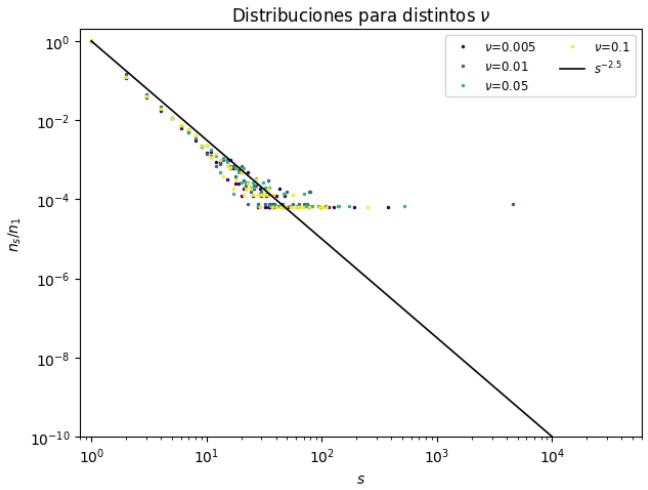


Figura 5. Distribuciones del número de unidades en función de su tamaño, obtenidas mediante Python para los valores del parámetro de fragmentación $\nu \in \{0{,}005,\ 0{,}01,\ 0{,}05,\ 0{,}1\}$.

C. Estimación y comparación del exponente

El exponente se estimó mediante un ajuste lineal en escala logarítmica, utilizando los valores positivos de n_s en el intervalo $2 \leq s \leq 30$. La Tabla II presenta los resultados de ambas implementaciones y la diferencia relativa entre sus estimaciones.

La diferencia relativa se calculó respecto al resultado de Python mediante

$$\Delta_\alpha = \frac{|\alpha_{\mathrm{C++}} - \alpha_{\mathrm{Python}}|}{\alpha_{\mathrm{Python}}} \times 100\,\%. \tag{9}$$

Las diferencias entre las estimaciones se encuentran entre aproximadamente 0,37 % y 2,17 %. El menor des-

Cuadro II. Exponentes estimados mediante C++ y Python y diferencia relativa entre los resultados.

ν	$\alpha_{\text{C++}}$	α_{Python}	Diferencia (%)
0.005	2.5953	2.605	0.37
0.010	2.5762	2.586	0.38
0.050	2.5686	2.514	2.17
0.100	2.5895	2.634	1.69

acuerdo se presenta para $\nu = 0{,}005$ y $\nu = 0{,}01$, mientras que el mayor corresponde a $\nu = 0{,}05$. Como referencia, el exponente teórico considerado en este trabajo es $\alpha = 2{,}5$. Los valores numéricos obtenidos se encuentran por encima de dicha referencia, aunque su cercanía varía según el parámetro y la implementación.

D. Desempeño computacional y conservación

La ejecución principal en Python utilizó 9 000 000 eventos y registró un tiempo de 28 406,1 segundos. Al finalizar, el sistema contenía 15 729 unidades y la suma de sus tamaños era 30 000, verificando la conservación de la fuerza total:

$$\sum_{i=1}^{M} s_i = N = 30\,000. \tag{10}$$

Por su parte, la implementación en C++ registró un tiempo de ejecución de 188,419 segundos para 900 000 000 pasos, con 120 000 muestras acumuladas. Estos tiempos corresponden a los procedimientos utilizados en cada implementación. Dado que Python analiza el estado final, mientras que C++ acumula muestras durante la ejecución, la comparación de rendimiento debe interpretarse teniendo en cuenta esta diferencia metodológica.

VI. DISCUSIÓN

Los resultados obtenidos muestran que las distribuciones de tamaños presentan un comportamiento decreciente, en el que las unidades de mayor tamaño son menos numerosas que las de menor tamaño. Este comportamiento es compatible con una distribución de ley de potencia, como la propuesta en el modelo de referencia. Sin embargo, los exponentes estimados no coinciden exactamente con el valor teórico $\alpha = 2{,}5$, lo que evidencia diferencias entre el comportamiento numérico y la predicción teórica.

La comparación entre las implementaciones en C++ y Python muestra una concordancia general en las estimaciones del exponente. Las diferencias relativas calculadas se encuentran entre 0,37 % y 2,17 %, lo que indica que ambos procedimientos producen resultados cuantitativamente cercanos. Las discrepancias pueden relacionarse con las diferencias metodológicas entre las implementaciones: Python obtiene la distribución a partir del estado final de la simulación, mientras que C++ acumula muestras durante su ejecución. Por tanto, los resultados no corresponden a procedimientos de muestreo idénticos.

En cuanto al parámetro de fragmentación ν, los exponentes estimados presentan variaciones moderadas para los cuatro valores estudiados. En Python, el exponente disminuye de 2,605 a 2,514 al pasar de $\nu = 0{,}005$ a $\nu = 0{,}05$, y posteriormente aumenta hasta 2,634 para $\nu = 0{,}1$. En C++, las estimaciones se mantienen en un intervalo más estrecho, entre 2,5686 y 2,5953. Estos resultados muestran que, dentro del conjunto de parámetros analizado, no se observa una variación estrictamente monótona del exponente con respecto a ν.

Por otra parte, la simulación en Python permitió verificar la conservación de la fuerza total del sistema, cuya suma permaneció en $N = 30\,000$ al finalizar el procedimiento. Esta comprobación respalda la correcta implementación de las operaciones de fragmentación y fusión desde el punto de vista de la conservación.

Finalmente, los tiempos de ejecución registrados muestran una diferencia importante entre las implementaciones. C++ completó su procedimiento en 188,419 segundos, mientras que Python registró 28 406,1 segundos. Aunque estos resultados evidencian un menor tiempo de ejecución para C++, la comparación debe considerarse con cautela, debido a que ambas implementaciones utilizaron diferentes cantidades de pasos y distintos procedimientos de acumulación de muestras.

A. Dificultades durante la implementación

Durante la implementación computacional del modelo se presentaron diversas dificultades, especialmente en el desarrollo del código en Python. Una de las principales consistió en estructurar adecuadamente el ciclo de simulación, de manera que la secuencia de operaciones representara correctamente los procesos de fragmentación y coalescencia. En particular, fue necesario implementar un mecanismo de selección aleatoria ponderada, en el cual la probabilidad de seleccionar cada grupo fuera proporcional a su tamaño, así como incorporar correctamente las probabilidades asociadas a cada uno de los procesos.

Otra dificultad importante estuvo relacionada con la actualización de la estructura de datos después de cada evento, especialmente en las operaciones de eliminación, modificación y creación de grupos. La implementación de estas operaciones requirió múltiples pruebas y ajustes para evitar inconsistencias en la evolución del sistema, lo que representó una inversión considerable de tiempo durante el desarrollo del código.

Asimismo, fue necesario incorporar mecanismos de verificación de la conservación de la fuerza total del sistema, comprobando que la suma de los tamaños de todos los grupos permaneciera constante durante la simulación. También se debió establecer un número apropiado de pasos para observar la evolución del modelo. Esta última consideración no había sido contemplada inicialmente y

se incorporó posteriormente como parte del proceso de revisión y ajuste del procedimiento, con apoyo de una herramienta de inteligencia artificial.

Finalmente, la representación e interpretación de los resultados también implicó dificultades, particularmente en la construcción de las distribuciones de tamaños en escala logarítmica y en la estimación del exponente α. Este último análisis se desarrolló de manera exploratoria, por lo que los valores obtenidos deben interpretarse con cautela y no como una caracterización estadística definitiva del comportamiento asintótico del modelo.

VII. REPOSITORIO Y DISPONIBILIDAD DEL CÓDIGO FUENTE

Con el propósito de organizar y facilitar la consulta de los recursos computacionales desarrollados, se creó un repositorio público en GitHub para el proyecto. Este contiene las implementaciones del modelo en C++ y Python, los cuadernos Jupyter, las figuras generadas durante las simulaciones, los resultados numéricos y los archivos de configuración y documentación.

El repositorio se encuentra disponible en el siguiente enlace:

`https://github.com/jomartinezdi-UNAL/Proyecto_G_Guerrillas`

La estructura del repositorio permite identificar por separado el código fuente, los notebooks de ejecución, las figuras, los resultados y los recursos necesarios para consultar y reproducir el procedimiento computacional.

VIII. CONCLUSIONES

- Se implementó computacionalmente un modelo estocástico de fragmentación y fusión en dos lenguajes de programación, C++ y Python, permitiendo estudiar la evolución de las unidades y sus distribuciones de tamaños.
- Las distribuciones obtenidas presentan un comportamiento decreciente compatible con una ley de potencia. Los exponentes estimados se encuentran por encima del valor teórico de referencia $\alpha = 2{,}5$, con valores entre 2,514 y 2,634 para Python, y entre 2,5686 y 2,5953 para C++.
- La comparación entre las implementaciones muestra una concordancia cuantitativa en la estimación del exponente, con diferencias relativas inferiores al 2,2 % para los cuatro valores de ν estudiados. Las discrepancias observadas deben interpretarse considerando que los procedimientos de acumulación de datos son diferentes.
- La variación del parámetro de fragmentación $\nu \in \{0{,}005,\ 0{,}01,\ 0{,}05,\ 0{,}1\}$ produjo cambios moderados en los exponentes estimados. No se identificó una relación monótona entre dicho parámetro y el exponente α dentro del intervalo analizado.
- La implementación en Python permitió verificar la conservación de la fuerza total del sistema, manteniéndose la suma de los tamaños igual a $N = 30\,000$ al finalizar la simulación.
- Finalmente, C++ presentó un menor tiempo de ejecución registrado que Python. No obstante, debido a las diferencias en el número de pasos y en los procedimientos de muestreo, esta comparación no constituye una evaluación controlada del rendimiento de ambos lenguajes.

[1] N. Johnson et al., *From old wars to new wars and global terrorism*, arXiv:physics/0506213, 2005.

[2] N. Johnson et al., *Universal patterns underlying ongoing wars and terrorism*, arXiv:physics/0605035, 2006.